Concurrencia, transacciones y bloqueos

Bases de Datos 2 · Práctica de dos horas · Oracle Autonomous Database

De qué se trata

- Una base de datos con un solo usuario es fácil. Todo lo difícil aparece cuando dos personas tocan el mismo dato al mismo tiempo.
- Hoy cada pareja abre dos sesiones sobre su propia base en la nube y provoca, a propósito, todo lo que puede salir mal.

Las dos horas

- 0:00 Conexión a la base y verificación de las dos sesiones
- 0:15 Transacción, ACID y el modelo multiversión de Oracle
- 0:35 Ejercicios 1 a 3: lectura consistente, bloqueo de fila, actualización perdida
- 1:05 Ejercicios 4 y 5: FOR UPDATE y su primer interbloqueo
- 1:25 Ejercicios 6 a 8: savepoint, serializable y la llave foránea sin índice
- 1:40 Retos en pareja y cierre

El entorno de hoy

Una Autonomous Database Always Free

- Creada en OCI, sin costo y sin tarjeta: 20 GB y 1 OCPU, suficiente de sobra para lo que vamos a hacer.
- Cada pareja tiene su propio usuario y su propio esquema, así que nadie bloquea a nadie por accidente.

Cómo se conectan

- Por Database Actions en el navegador, con usuario y contraseña. Sin instalar nada.
- Necesitan DOS ventanas: una normal y otra de incógnito, o dos navegadores distintos.
- **En cada hoja de trabajo hay que APAGAR el autocommit. Si queda encendido, ningún ejercicio funciona.**

La comprobación de arranque

```
SELECT SYS_CONTEXT('USERENV','SID') FROM dual;
```

- **Los dos números deben ser distintos. Si son iguales, no hay dos sesiones y no hay práctica.**

Qué es una transacción

La definición

- Un conjunto de operaciones que la base trata como una sola: o se aplican todas, o no se aplica ninguna.
- Empieza sola con la primera sentencia que modifica datos, y termina cuando usted dice COMMIT o ROLLBACK.

ACID, en una línea cada una

- Atomicidad — todo o nada
- Consistencia — la base pasa de un estado válido a otro estado válido
- Aislamiento — cada transacción trabaja como si estuviera sola
- Durabilidad — lo confirmado sobrevive a una caída del servidor

El ejemplo de siempre

- Una transferencia son dos UPDATE. Si el servidor se cae entre los dos, el dinero no puede desaparecer. De eso responde la atomicidad.
- **Y el aislamiento es el que nos va a ocupar las dos horas: es el único de los cuatro que se puede graduar.**

Cómo se controla

Las tres instrucciones

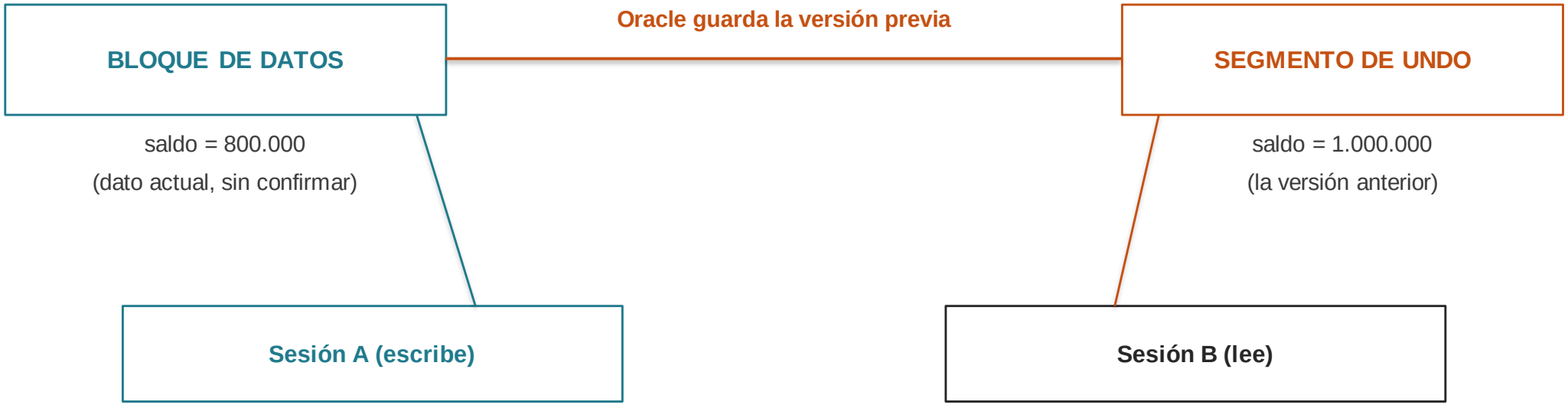
```
COMMIT;                      -- confirma y libera los bloqueos
ROLLBACK;                    -- deshace todo desde el inicio
SAVEPOINT nombre;  ...  ROLLBACK TO SAVEPOINT nombre;
```

- El savepoint permite deshacer una parte sin perder el resto. La transacción sigue abierta.

Las trampas

- **Toda sentencia DDL hace COMMIT implícito, antes y después. Un CREATE TABLE en medio de su transacción confirma todo lo que llevaba pendiente.**
- Cerrar la sesión limpiamente hace COMMIT; que se caiga la conexión hace ROLLBACK. No confíe en ninguno de los dos.
- **Una transacción abierta retiene bloqueos y consume undo. Nunca deje una esperando a que un usuario conteste una pantalla.**
- **Ejercicio 6 — savepoint y el COMMIT del DDL**

El modelo de Oracle: multiversión



B no espera a A, y tampoco ve su cambio a medias: lee la versión anterior desde el undo

`Los lectores`	no bloquean a los escritores
`Los escritores`	no bloquean a los lectores
`Los escritores`	sí se bloquean entre sí, sobre la misma fila
`En Oracle`	no existe la lectura sucia: es imposible ver un dato sin confirmar

Las cuatro anomalías de concurrencia

Anomalía	Qué ocurre	¿Puede pasar en Oracle?
Lectura sucia	leer un dato que otra transacción aún no confirmó	NUNCA, en ningún nivel
Lectura no repetible	leer dos veces lo mismo y obtener valores distintos	Sí, en READ COMMITTED
Lectura fantasma	repetir una consulta y encontrar filas nuevas	Sí, en READ COMMITTED
Actualización perdida	dos sesiones escriben y una pisa a la otra sin dejar rastro	Sí, y es la más peligrosa

Las tres primeras las controla el nivel de aislamiento. La cuarta no: la evita el programador, y por eso es la que causa daños de verdad

La actualización perdida no produce ningún error. La base queda consistente, pero con un valor equivocado

Ejercicios 1, 2 y 3 de la práctica

Los niveles de aislamiento en Oracle

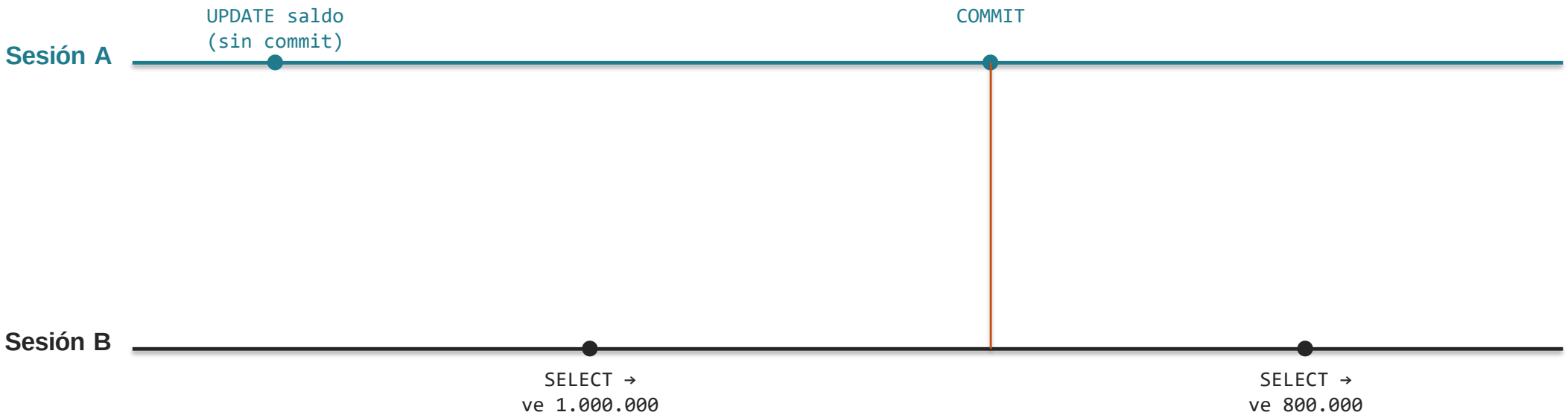
Nivel	Qué garantiza	Cuándo se usa
READ COMMITTED (por defecto)	cada sentencia ve una foto del momento en que empezó ESA sentencia	el 99 % de las aplicaciones
SERIALIZABLE	toda la transacción ve la foto del momento en que empezó LA TRANSACCIÓN	informes que deben cuadrar, y procesos que releen
READ ONLY	igual que serializable, pero no permite escribir nada	reportes largos, cierres contables

```
SET TRANSACTION READ ONLY;
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

En SERIALIZABLE, si otra sesión modifica una fila que usted también quiere cambiar, recibe ORA-08177 y le toca reintentar la transacción completa

Ejercicio 7 de la práctica

Ejercicio 1 · Los lectores no se bloquean



B leyó dos veces y obtuvo respuestas distintas: eso es una lectura NO REPETIBLE, y READ COMMITTED la permite a propósito

Lo importante: B nunca esperó, y nunca vio el valor sin confirmar

Ejercicio 2 · Bloqueo de fila



El bloqueo es de FILA, no de tabla: otra sesión puede modificar cualquier otra fila sin esperar

Oracle no tiene tiempo límite por defecto: B habría esperado indefinidamente. Por eso existen NOWAIT y WAIT n

La actualización perdida

Cómo ocurre

- A lee el saldo: 500.000. B lee el mismo saldo: 500.000. A calcula en la aplicación y escribe 400.000. B calcula y escribe 450.000.
- **El resultado final es 450.000. Los 100.000 que retiró A desaparecieron, sin ningún error y sin ningún rastro.**

Por qué el bloqueo de fila no la evita

- B esperó a que A confirmara, sí. Pero cuando se destrabó, escribió el valor que había calculado ANTES, con datos ya viejos.
- La clave está en si el UPDATE recalcula sobre el valor actual —"saldo = saldo - 100000"— o escribe un número fijo traído de la aplicación.

Las dos defensas

- Pesimista: SELECT ... FOR UPDATE. Se bloquea la fila desde que se lee. Seguro, y hace esperar a los demás.
- **Optimista: una columna de versión y un WHERE version = la que leí. Nadie espera; si el UPDATE afecta cero filas, hubo choque y se reintenta.**
- **Ejercicio 3 — las dos estrategias, una detrás de otra**

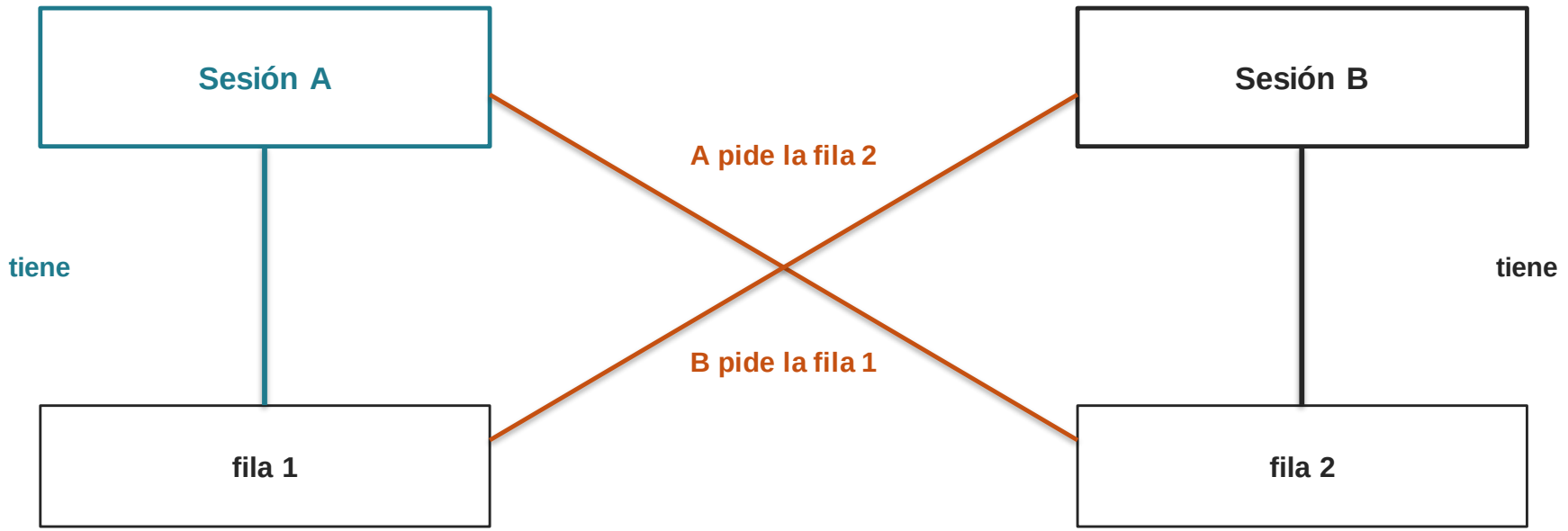
Las variantes de SELECT FOR UPDATE

FOR UPDATE	**Espera lo que haga falta** el comportamiento por defecto: puede esperar para siempre
FOR UPDATE NOWAIT	**Falla al instante** ORA-00054. Ideal en una pantalla: avísele al usuario, no lo deje colgado
FOR UPDATE WAIT 5	**Espera 5 segundos** ORA-30006 al vencer. El término medio razonable
FOR UPDATE SKIP LOCKED	**Ignora lo ocupado** devuelve solo las filas libres. Es como se reparte una cola de trabajos

```
SELECT id_trabajo FROM trabajo WHERE estado = 'PENDIENTE'
ORDER BY id_trabajo FETCH FIRST 3 ROWS ONLY FOR UPDATE SKIP LOCKED;
```

Con SKIP LOCKED, diez procesos pueden vaciar la misma cola sin coordinador, sin bloqueos y sin repetir trabajo

Ejercicio 5 · El interbloqueo




```
ORA-00060: deadlock detected while waiting for resource
```

Oracle detecta el ciclo en segundos y sacrifica UNA de las dos sentencias. No deshace la transacción entera: la víctima queda abierta y debe decidir

Cómo se evita: acceder siempre a las filas en el mismo orden. Si las dos hubieran empezado por la fila de menor id, no habría ciclo posible

El bloqueo que nadie espera

El escenario

- MOVIMIENTO tiene una llave foránea hacia CUENTA, y esa columna no está indexada. La sesión A inserta un movimiento de la cuenta 2 y no confirma.
- La sesión B intenta borrar la cuenta 3, que no tiene ningún movimiento, y se queda esperando.

Por qué pasa

- Para comprobar que ninguna fila hija referencia al padre que se borra, Oracle tendría que recorrer la tabla hija entera. En vez de eso, toma un bloqueo de tabla sobre el hijo.
- **Con el índice, la comprobación es una búsqueda puntual y el bloqueo de tabla desaparece.**

La regla

- **Toda llave foránea cuyo padre se borre o se actualice necesita índice. En bases OLTP, casi todas.**
- Es una de las causas más frecuentes de bloqueos inexplicables en producción, y una de las más fáciles de arreglar.
- **Ejercicio 8 — provocarlo, y arreglarlo creando el índice**

Cómo se diagnostica en caliente

La consulta que responde “¿quién me bloquea?”

```
SELECT sid, username, status, blocking_session,
       event, seconds_in_wait
  FROM v$session WHERE username LIKE 'P%';
```

- La sesión con BLOCKING_SESSION lleno es la víctima; ese número es el SID de quien la tiene detenida.

Qué está bloqueado

- V$LOCK muestra el tipo: TX es bloqueo de transacción, es decir de fila. TM es bloqueo de tabla, el que aparece cuando falta el índice de una foránea.

El detalle que confunde a todos

- **Quien bloquea suele aparecer INACTIVE. Ya terminó su sentencia: lo que falta es el COMMIT que nunca hizo. Por eso se consulta PREV_SQL_ID y no SQL_ID.**
- **03_diagnostico.sql — para usar durante los ejercicios**

Buenas prácticas

Sobre la duración

- Transacciones cortas. Abra lo más tarde posible y confirme lo antes posible.
- Nunca deje una transacción abierta esperando que un usuario conteste algo en pantalla. Ese es el origen del bloqueo que dura toda la tarde.

Sobre el orden

- Acceda siempre a las tablas y a las filas en el mismo orden en toda la aplicación. Es la defensa más barata contra los interbloqueos.

Sobre el diseño

- Indexe las llaves foráneas.
- Use FOR UPDATE cuando la espera sea aceptable, y control de versión cuando no lo sea.
- No suba el nivel de aislamiento por si acaso: SERIALIZABLE trae ORA-08177 y obliga a escribir lógica de reintento.

Y una regla de oro

- **Si una transacción se puede reintentar sin daño, prefiera fallar rápido y reintentar antes que bloquear y esperar.**

El guion de la práctica

#	Ejercicio	Qué se observa	Tiempo
1	Lectura consistente	los lectores no esperan	10 min
2	Bloqueo de fila	B se detiene sin aviso	10 min
3	Actualización perdida	pesimista y optimista	15 min
4	FOR UPDATE	NOWAIT, WAIT, SKIP LOCKED	15 min
5	Interbloqueo	ORA-00060 en vivo	10 min
6	Savepoint y DDL	el COMMIT que nadie pidió	8 min
7	SERIALIZABLE	ORA-08177	8 min
8	Foránea sin índice	bloqueo de tabla	8 min

Los archivos 02_sesion_A.sql y 02_sesion_B.sql están sincronizados por PASOS: se alterna entre las dos ventanas, nunca se corre uno completo

Lo que hay que llevarse

Seis frases

- En Oracle los lectores nunca se bloquean, y jamás ven un dato sin confirmar.
- Los escritores sí se bloquean entre sí, y solo sobre la misma fila.
- La actualización perdida no da error: la evita el programador, no el motor.
- El interbloqueo se previene con un orden de acceso consistente, no con más bloqueos.
- Toda llave foránea cuyo padre se borre necesita índice.
- **Una transacción larga es un problema de diseño, no de rendimiento.**

Autoevaluación

- ¿Por qué en Oracle es imposible una lectura sucia?
- ¿En qué caso el bloqueo de fila NO evita una actualización perdida?
- ¿Cuándo prefiere NOWAIT y cuándo SKIP LOCKED?
- ¿Qué hace Oracle exactamente al detectar un interbloqueo?
- ¿Por qué la sesión que bloquea aparece como INACTIVE?